

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

PROJECT ASSIGNMENT

Coverage-Driven Verification of a Multi-Cycle ALU

INSTRUCTOR

Not identified in the supplied materials

Contents

1 Objective and protocol 2

2 Interface task 2

3 Transaction task 2

4 Driver task 2

5 Monitor task 2

6 Reference-model task 2

7 Scoreboard and top-level task 3

8 Coverage and test-plan task 3

9 Test execution and ranking task 3

1 Objective and protocol

Construct a class-based SystemVerilog verification environment for the supplied packetized multi-cycle ALU. Use the default widths of eight-bit operand buses, 32-bit full operands, a 16-bit result bus and 64-bit full results. Adapt the environment rather than treating each bus beat as a complete operation. Verify no-op, unsigned add, bitwise AND, bitwise XOR, unsigned multiply and software reset. Respect the active-low physical reset and software reset opcode 111.

2 Interface task

Create input and output interfaces with separate DUT, driver and monitor directions. The input side must include ready, operand_valid, operand_last, op, a and b. The output side must include result_valid, result_last, result_rst and result. Share clock and reset correctly. Connect every width parameter and signal to the supplied ALU instance. The recorded interface task annotation is five points.

3 Transaction task

Store full operands in input objects and full results in output objects, independently of bus widths. Add clk_gap constrained from zero to fourteen. Preserve gap and arithmetic fields in transaction copies; arithmetic comparison must not depend on the time spacing between identical inputs. Provide output reset and interruption state and copy them completely. Define interruption as cancellation rather than a valid arithmetic answer. The recorded transaction task annotation is ten points.

4 Driver task

Provide a mailbox/event-connected driver that sends least-significant chunks first, asserts operand_valid for each beat and operand_last for the final beat. Wait for ready for ordinary operations and apply the requested idle-clock gap. Drive values before the DUT sampling edge. Software reset must be sendable while ready is low, using one accepted command cycle. Allow legal resets during calculation or output collection and distinguish consecutive commands. The recorded driver task annotation is twenty points.

5 Monitor task

Implement independent input and output monitors. Input reconstruction must use accepted operand beats and exclude busy/reset clocks from idle-gap measurement. Check maximum frame length and assemble complete operands before calling the model. Recognize software reset despite ready being low. Output reconstruction must use result_valid and result_last, detect interruption by result_rst, and reject unexplained truncation. Recognize reset notifications before the first output beat and during a frame. The recorded monitor task annotation is twenty-five points.

6 Reference-model task

Compute unsigned full-width addition and multiplication and bitwise AND/XOR from reconstructed operands. Produce zero with result_rst asserted for reset; ordinary results have result_rst clear. No-op must not enqueue an arithmetic result. Keep idle spacing out of the arithmetic reference. The recorded model task annotation is ten points.

7 Scoreboard and top-level task

Connect the model and output monitor using ordered mailboxes. Check numeric results and reset flags. A legal reset may cancel one pending arithmetic reference, but the actual reset must still be zero and match the following reset reference. Maintain separate completed, aborted and failed counters. Report unconsumed queues at the end. Instantiate the new DUT with the appropriate parameter and port mapping and make the selected test agree with the recorded test name. The recorded test-instantiation annotation is five points.

8 Coverage and test-plan task

Provide input operation, operand and idle-gap coverpoints. Use six defined operation bins and treat undefined opcodes as illegal. Group gap values into zero, one through eight, and nine through fourteen. Define disjoint low/mid/high unsigned operand ranges and qualify numeric sampling with arithmetic operations. On the output side, cover valid numeric results, presence/absence of reset, and interruption of a pending computation. Exclude reset and aborted values from numeric-result bins. Use at-least-ten hit goals.

Write a spreadsheet-style plan with Section, Title, Description, Link, Type, Weight and Goal columns. Link every coverage requirement to its implemented coverpoint and provide the workbook representation needed by the coverage tool. Track both gap diversity and reset during calculation/result collection explicitly. Set the documented coverage goals to 100 percent. The recorded test-plan annotation is twenty points.

9 Test execution and ranking task

Provide unrestricted random, mid-range, high-range and mixed-profile tests. Install range blueprints before generation. Run tests with a fixed recorded seed, save individual coverage databases, merge functional coverage and rank tests against the selected functional goals. Distinguish overall/tool coverage from weighted test-plan coverage and report any remaining bins. Retain scoreboard summaries, coverage output and ranking evidence. The recorded ranking-task annotation is twenty points.

Do not claim passing arithmetic comparisons for canceled operations. A 100-percent coverage value means the specified bins and hit requirements were exercised; it does not prove completeness of the specification or correctness over all possible input sequences. Keep all source, build/run instructions, test profiles and reproducible results in the code directory, and include complete architecture, protocol, transaction, monitor, model, scoreboard and coverage explanations in the technical report.